

INDUZIONE STRUTTURALE

Principio $\rightarrow$ Base definire/dimostrare una proprietà sugli elementi base dell'insieme (non definiti in funzione di altri elementi)

Passo induttivo

Hp. induttiva assumere la proprietà sugli elementi dell'insieme

Passo definizione/dimostrazione della proprietà sugli elementi composti:

la proprietà è assunta per tutti i componenti immediati

$$x \in \mathbb{N}$$

Base: $0 \in X$

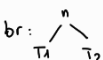
Passo: se $x \in X$ allora $x+3 \in X$

dim. $x \in 3\mathbb{N}$

Alberi binari BT con nodi $\mathbb{N}$

Base: $n \in \mathbb{N}$ e un BT ($n \in BT$) ... $N \in BT$

Passo: se $T_1, T_2 \in BT$ allora $br(n, T_1, T_2) \in BT$



Foglie: conta il numero di foglie di $T \in BT$

$$\begin{cases} foglie(n) = 1 \\ foglie(br(n, T_1, T_2)) = foglie(T_1) + foglie(T_2) \end{cases}$$

Nodi: conta il numero di nodi di $T \in BT$

Branch: conta il numero di diramazioni di $T \in BT$

$$\begin{cases} branch(n) = 0 \\ branch(br(n, T_1, T_2)) = 1 + branch(T_1) + branch(T_2) \end{cases}$$

$$\forall T \in BT \quad foglie(T) = branch(T) + 1$$

Base $n \in \mathbb{N}$ $foglie(n) = 1 + 0 = 1 + branch(n)$
 $branch(n) = 0$

Passo $br(n, T_1, T_2) \in BT$ $T_1, T_2 \in BT$

Hp. induttiva $\rightarrow$ assicurare la proprietà per i componenti immediati:
 $foglie(T_1) = branch(T_1) + 1$
 $foglie(T_2) = branch(T_2) + 1$

$$\begin{aligned} foglie(br(n, T_1, T_2)) &= foglie(T_1) + foglie(T_2) \\ &= branch(T_1) + 1 + branch(T_2) + 1 \end{aligned}$$

Hp. induttiva

$$\begin{aligned} &= (branch(T_1) + branch(T_2) + 1) + 1 \\ &= branch(br(n, T_1, T_2)) + 1 \end{aligned}$$

$$(P) \rightarrow aa | bb | bPb | aPa$$

def. insieme SP : $aa, bb \in SP$

se $x \in SP$ allora $axa \in SP$
 $bxb \in SP$

Dimostriamo: $\forall x \in SP \quad \exists yz \in \{a, b\}^* \quad x = yz \text{ e } y = reverse(z)$

$\downarrow$
 $|x|$ pari

Base: $x = aa, y = a, z = a$
 $x = yz = aa \quad y = reverse(z) = a$

$x = bb, y = b, z = b$
 $x = yz = bb \quad y = reverse(z) = b$

Passo: $x \in SP$

$\hookrightarrow axa \stackrel{x'}{\sim} x$ Hp. ind. $\exists yz. x = yz \quad y = reverse(z)$
dobbiamo dim. $\exists y'z'. x' = ay'a$
 $y' = reverse(z')$

$$y' = ay \quad z' = za \Rightarrow x' = az = ayza = y'za'$$

$$\text{reverse}(z') = \text{reverse}(za) = a \text{ reverse}(z)$$

$$= ay = y'$$

↳ bxb analogo

Grammatica espressioni aritmetiche

$$E \rightarrow n \mid E + E \mid E * E \quad n \in \mathbb{N}$$

Base $n \in \text{Exp} \quad (n \in \mathbb{N})$

Passo se $e_1, e_2 \in \text{Exp}$ allora

$$e_1 + e_2 \in \text{Exp}$$

$$e_1 * e_2 \in \text{Exp}$$

op = conta il numero di operatori in una espressione

$$\text{op}(n) = 0$$

$$\text{op}(e_1 + e_2) = \text{op}(e_1 * e_2) = \text{op}(e_1) + \text{op}(e_2) + 1$$

val = conta il numero di valori in una espressione

$$\text{val}(n) = 1$$

$$\text{val}(e_1 + e_2) = \text{val}(e_1 * e_2) = \text{val}(e_1) + \text{val}(e_2)$$

$$\text{Dim. } \forall e \in \text{Exp. } \text{val}(e) = \text{op}(e) + 1$$

Base $n \in \text{Exp} \quad \text{val}(n) = 1 = 1 + 0 = 1 + \text{op}(n) \quad \checkmark$

Passo $e_1, e_2 \in \text{Exp} \quad e_1 + e_2$ (analogo $e_1 * e_2$)

dobbiamo dim. $\text{val}(e_1 + e_2) = \text{op}(e_1 + e_2) + 1$

$$\text{Hp. ind. } \begin{cases} \text{val}(e_1) = \text{op}(e_1) + 1 \\ \text{val}(e_2) = \text{op}(e_2) + 1 \end{cases}$$

$$\text{val}(e_1 + e_2) = \text{val}(e_1) + \text{val}(e_2) = \text{op}(e_1) + 1 + \text{op}(e_2) + 1$$

$$= (\underbrace{\text{op}(e_1) + \text{op}(e_2) + 1}_{\text{def op}}) + 1$$

$$= \text{op}(e_1 + e_2) + 1 \quad \checkmark$$

COMPOSIZIONALITÀ

Il significato di op: costruito deve essere funzione del significato delle componenti immediate

SISTEMI DI TRANSIZIONE

- matematicamente precisi
- concisi
- Permetto di dare semantica in modo induttivo sulla struttura della sintassi
- questo avviene mediante un insieme di regole
- metodo di specifica generale → permette astrazione

Def. $\langle T, \rightarrow \rangle$ è un sistema di transizione dove

Γ insieme di configurazioni/stati $t \in T$

(descrizione formale dello stato della macchina)

→ relazione binaria di transizione

$$\rightarrow \subseteq T \times T$$

Notazioni: $(t_1, t_2) \in \rightarrow$ viene denotato $t_1 \rightarrow t_2$

$$t_0 \rightarrow^* t_n \Leftrightarrow \exists t_1, t_2 \dots t_{n-1} \text{ t.c.}$$

$$t_0 \rightarrow t_1 \rightarrow t_2 \rightarrow \dots \rightarrow t_{n-1} \rightarrow t_n$$

Sistema di transizione terminale

$\langle T, T, \rightarrow \rangle$ dove

$\langle T, \rightarrow \rangle$ è un sistema di transizione

$T \subseteq T$ insieme di configurazioni terminali

$$\text{t.c. } \forall t \in T. \nexists t' \in T$$

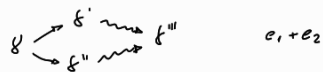
$$t \rightarrow t'$$

Grammatica

$G = \langle V, T, P, S \rangle \rightarrow \langle T, \rightarrow \rangle^T$ sistema di transizione

$$T = P \quad \frac{A \rightarrow \alpha}{P A \delta \rightarrow P \alpha \delta} \equiv (A \rightarrow \alpha) \rightarrow (P A \delta \rightarrow P \alpha \delta)$$
$$T = A \rightarrow \alpha \quad \begin{matrix} \alpha \in T^* \\ S \rightarrow \alpha \in T^* \end{matrix}$$

Non determinismo



CATEGORIE SINTATTICHE

↳ manipolare dati (espressioni)

↳ creazione/manipolazione di legami (dichiarazione) *puntatori*

↳ trasformare stack (comandi)

Ambiente

Insieme dei
legami:

id → oggetti

Memoria

permette di descrivere gli effetti di
una trasformazione

ESPRESSIONI → denotano valori
→ vengono valutate per restituire un valore
→ sono equivalenti se denotano lo stesso valore
(sono uguali se sono sintatticamente uguali)

DICHIARAZIONI

- Denotano richieste di modifica degli ambienti
- Vengono elaborate per ottenere le trasformazioni dell'ambiente
- Dichiarazioni equivalenti se vengono elaborate nello stesso ambiente

COMANDI

- Denotano richieste di modifica della memoria
- Vengono eseguiti per ottenere la trasformazione della memoria corrispondente